Ift3395_Elbroumi _Loungnarath

Aya Elbroumi 20267991
Alix Loungnarath 20274695

Rapport sur la compétition Kaggle 2

1. Introduction

L'objectif de ce laboratoire était de développer un modèle d'apprentissage automatique capable d'évaluer la gravité de la rétinopathie diabétique sur une échelle de 0 à 5 à partir de photographies du fond de l'œil. Pour résoudre ce problème de classification multiclasse, nous avons exploré les données d'entraînement et effectué quelques manipulations sur ces dernières. Ensuite, nous avons divisé nos données en deux ensembles (un ensemble de test et un ensemble de validation) puis sélectionné quelques modèles que nous avons entraînés avec différents ensembles d'hyperparamètres pour finalement sélectionner celui avec la meilleure accuracy. Finalement, nous avons utilisé notre meilleur modèle afin de produire nos prédictions des labels de l'ensemble de test.

2. Pré-traitement des données

Pour le jalon 1:

Nous avons commencé par explorer nos données en analysant la moyenne de chaque couleur (R, G, B) selon chaque classe. Nous avons remarqué que la moyenne de bleu et de rouge varie selon la classe, alors que la quantité de vert ne varie pas beaucoup. Nous l'avons tout de même gardée comme caractéristique puisque, bien qu'elle varie moins que les autres, son écart-type est beaucoup plus petit. Il est donc possible que cette couleur contienne tout de même des informations importantes. Nous avons également vérifié la présence de données manquantes (il n'y en avait aucune, donc nous n'avons pas dû les traiter) ainsi que le nombre d'occurrences par classe et la forme de nos données et de nos labels.

Puisque les labels étaient sous forme d'array à une dimension, nous avons extrait chaque valeur de son array. Nous avons ensuite remarqué que les classes s'étalent de 0 à 4 et qu'elles sont un peu débalancées, surtout pour la classe 0 (surreprésentée) et la classe 4 (sous-représentée), mais nous n'avons rien fait dans ce jalon pour y remédier.

Pour ce qui est des images, nous les avons d'abord aplaties. Leur forme est donc passée de (1080, 28, 28, 3) à (1080, 2352). Cette étape est utile puisqu'il est plus facile d'avoir un vecteur d'une dimension pour chacune des données. Ça permet de simplifier les modèles que nous allions devoir implémenter.

Ensuite, nous avons appliqué une mise à l'échelle et une normalisation z-score sur chacune des couleurs. Ainsi, les caractéristiques de nos données suivent une loi normale et la valeur absolue de chacune de ces valeurs est entre 0 et 1. Cela nous permettra d'entraîner des modèles avec une meilleure précision puisque nos données sont maintenant uniformes.

Ajout pour le jalon 2 :

Nous avons un peu modifié le prétraitement pour le jalon deux puisque nous avons décidé de nous concentrer sur l'implémentation d'un CNN en utilisant la bibliothèque PyTorch. Nous avons donc retiré l'aplatissage de nos données puisque celui-ci est effectué directement dans notre modèle. De plus, nous avons décidé d'augmenter le contraste entre nos couleurs et nous avons transposé nos données dans le format utilisé par PyTorch ([N, H, W, C] -> [N, C, H, W]). Finalement, nous avons augmenté la taille de nos données d'entraînement, en espérant ainsi obtenir de meilleurs résultats avec notre modèle. Pour ce faire, nous avons défini une banque de transformations possibles et, pour chaque image, nous avons produit un certain nombre de nouvelles données transformées en choisissant les transformations de manière aléatoire. Le nombre de nouvelles données produites par chaque image dépendait du poids de la classe de cette donnée. Cette technique nous a permis de balancer nos données pour chaque classe.

3. Algorithmes

Pour le jalon 1 :

Les premiers algorithmes que nous avons utilisés dans le jalon 1 sont Parzen (avec 21 radius différents entre 0.1 et 10.1, en multipliant par 2 la valeur du rayon si aucun voisin n'était trouvé) et KNN (avec un nombre de voisins de 1 à 27) avec la distance de Minkowski. Comme nous n'arrivons pas à battre la baseline forte avec ces algorithmes, nous avons cherché un peu plus loin. En effet, nous avons remarqué que, comme la moyenne de chaque couleur diffère selon la classe, il serait probablement efficace d'utiliser un algorithme de classification linéaire. Nous avons donc choisi d'implémenter l'algorithme de régression logistique avec régularisation L2 puis nous l'avons utilisé pour trouver un modèle par classe permettant de séparer cette classe du reste des autres classes. C'est l'algorithme qui nous a donné la meilleure accuracy pour le jalon un et deux et qui nous a permis de battre les deux baselines.

Pour le jalon 2:

Pour le jalon 2, nous étions déterminées à utiliser le modèle CNN puisque c'est le modèle généralement utilisé dans la classification d'images. Nous avons donc construit nos trois loaders (celui d'entraînement, de validation et de test), puis nous avons défini les différentes couches de notre modèle ainsi que sa fonction d'entraînement et de test. Nous avons utilisé l'entropie croisée comme fonction de perte et l'algorithme Adam pour l'ajustement des poids de notre réseau. Nous avons aussi implémenté NN-séquentiel avant de finalement se concentrer sur le CNN. Cet algorithme n'a pas su générer une accuracy plus haute que la régression logistique, donc nous avons décidé de ne pas le garder dans notre code final. Nous avons décidé de l'implémenter parce qu'elle semblait plus rapide, efficace à réaliser et adaptée aux données aplaties.

4. Méthodologie

Pour le jalon 1:

1. Nous avons mélangé les données et avons utilisé les 900 premières comme ensemble d'entraînement et le reste comme ensemble de validation.

2. Pour les modèles KNN et Parzen, nous avons entraîné ces modèles sur différents choix d'hyperparamètres et avons gardé le modèle avec la meilleure accuracy sur l'ensemble de validation.
3. Pour le modèle de régression logistique, nous avons utilisé la descente de gradient avec régularisation L2 afin de trouver les poids qui permettent de mieux séparer les données d'une classe avec les autres (loss la plus petite sur l'ensemble de validation).
4. Parmi tous nos modèles, c'est celui de régression logistique qui nous a permis d'obtenir la meilleure accuracy sur notre ensemble de validation, donc nous l'avons choisi pour prédire les labels de notre ensemble de test.

Pour le jalon 2 :

1. Nous avons séparé nos données avec la méthode de scikit-learn "train_test_split"
2. Pour NN-séquentiel un processus similaire a été fait en faisant une recherche par grid search avec des hyper paramètre de learning rate, epoch, dropout rate et hidden size choisis en faisant des tests d'essais erreur au préalable pour voir quelles potentielles valeurs hyperparamètres pourrait être avantageuse. Nous avons ensuite entraîné le modèle avec la configuration optimale sélectionnée avec le grid search.
3. Nous avons abandonné le modèle NN-séquentiel de PyTorch pour se concentrer sur CNN qui en théorie devrait mieux traiter nos données. Ce choix a été basé sur l'accuracy très basse rapportée par Kaggle (voir tableau 3).
4. Pour l'implémentation de CNN, nous avons mis en place une stratégie de régularisation, nous avons augmenté et balancé nos données et nous avons désactivé 50 % des sorties de la couche cachée avant la dernière couche linéaire (nn.Droupout(0.5)).
5. Une fois notre modèle CNN mis en place, nous l'avons testé plusieurs fois avec différentes valeurs pour le taux d'apprentissage et la valeur standard de 64 pour le batch size. Nous ajustions le taux d'apprentissage en fonction de nos observations sur le changement de l'accuracy de notre modèle sur les données de validation au cours des époques. Les étapes 4 et 5 ont donc été répétées plusieurs fois jusqu'à l'atteinte d'un résultat satisfaisant (oscillations faibles et diminution visible de l'accuracy).
6. Pour chaque époque, entraîner le modèle sur l'ensemble d'entraînement puis le tester sur l'ensemble de validation. Si l'accuracy obtenue est la plus haute observée jusqu'à maintenant, enregistrer notre modèle comme meilleur modèle.
7. Finalement, nous avons utilisé le meilleur modèle qui a été sauvegardé pour prédire les données de test.

5. Résultats

Pour le jalon 1:

Tableau 1: Précision des modèles pour le jalon 1

Modèle	Accuracy	Meilleurs hyperparamètres
K-NN	0,5000	k: 5
Parzen	0,4889	Radius: 0,1
Régression logistique	0,561	

Tableau 2: Précisions du modèle de régression logistique

Précision sur l'ensemble de validation	Précision sur l'ensemble de test publique	Précision sur l'ensemble de test privé
0,5611111111111111	0,4850	0,5450

Pour le jalon 2:

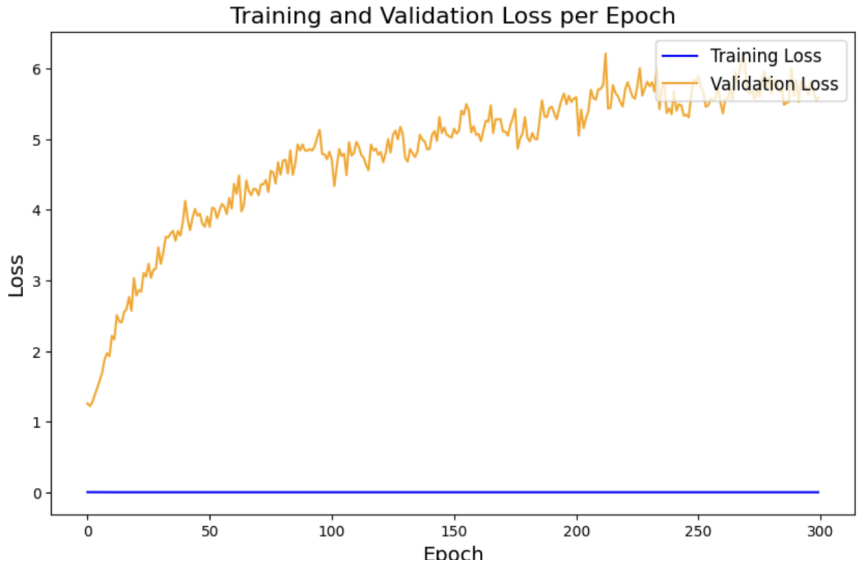
Tableau 3: Précision des modèles pour le jalon 2

Modèle	Accuracy	Meilleurs hyperparamètres
CNN	~ 0,55	Learning Rate:: 0.000001 Batch size: 64 Epoch : ~ 100
NN-séquentiel	Avant les tests cachés : ~ 0,5833 Après les tests cachés (score public) : 0,375	Learning Rate: 0,000800 Dropout Rate: 0,4 Hidden Size: 200 Epoch : 200

*****Les résultats pour le meilleur modèle de CNN et NN-séquentiel varient légèrement à chaque ré-entraînement

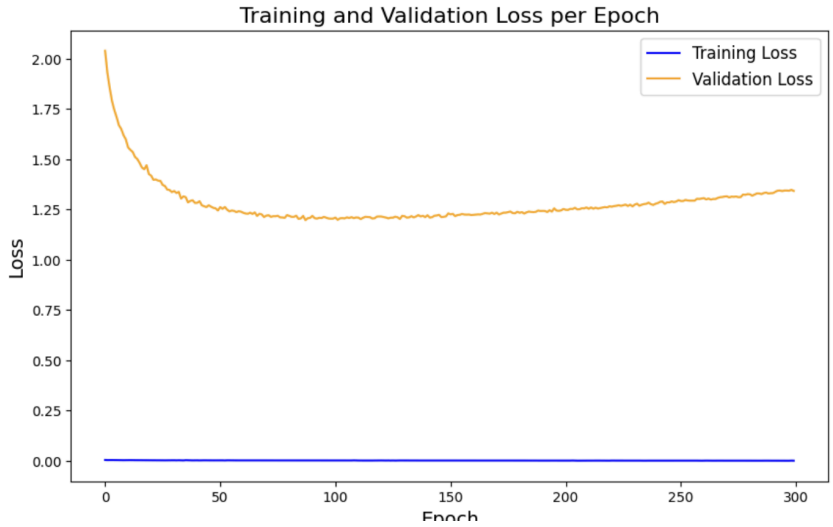
Tableau 4: Précisions du modèle CNN

Précision sur l'ensemble de validation	Précision sur l'ensemble de test publique	Précision sur l'ensemble de test privé
~ 0.55	0,5350	0,4750



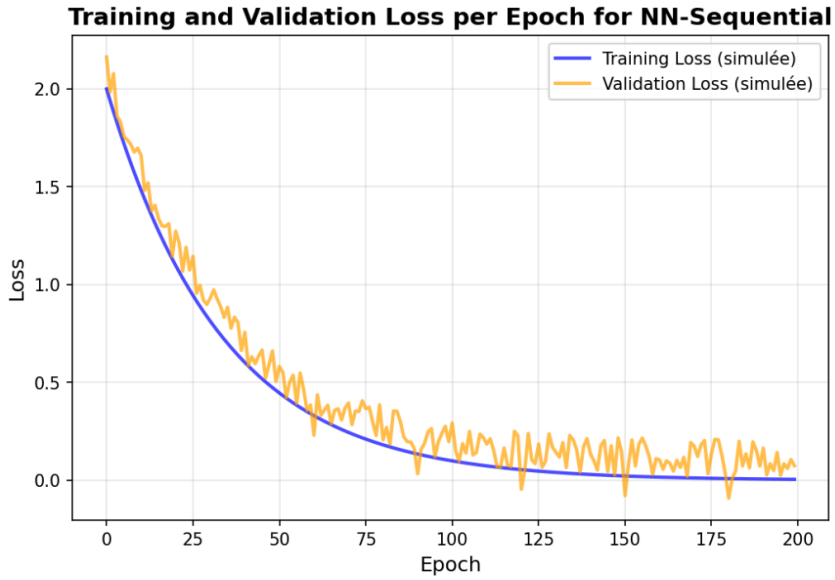
Learning rate: 0.0001
Batch Size: 64
Nombre d'époques: 300

Figure 1 : On remarque que la perte oscille fortement et augmente au fil des époques pour CNN dans ce cas-ci. Le loss de notre ensemble d'entraînement est toujours très près de 0,0 ce qui montre un surapprentissage clair de nos données.



Learning rate: 0.000001
Batch Size: 64
Nombre d'époques: 300

Figure 2: On remarque que la perte diminue jusqu'à un certain point et les oscillations sont très légères pour CNN dans ce cas-ci. Le loss de notre ensemble d'entraînement est toujours très près de 0,0 ce qui montre un surapprentissage clair de nos données.



Learning Rate: 0,000800
Dropout Rate: 0,4
Hidden Size: 200
Epoch : 200

Figure 3 : On remarque que pour NN-séquentiel, la perte diminue au fil des époques alors que l'accuracy atteint son maximum assez tôt pour ensuite légèrement osciller. Le loss de notre ensemble d'entrainement finit très près de 0,0 ce qui indique un léger surapprentissage.

6. Discussion

Notre approche comporte beaucoup d'inconvénients. En effet, les graphiques démontrent un surapprentissage fort. Voici une liste d'améliorations à apporter à notre approche :

1. Afficher le graphique dès le début pour éviter de ne pas remarquer le surapprentissage.
2. Simplifier notre modèle et ajouter une méthode de régularisation à notre optimiseur.
3. Utiliser un modèle plus simple (soit enlever des couches et diminuer les tailles de sorties de notre CNN, ou simplement utiliser des modèles plus simples comme la régression logistique qui avait bien fonctionnée)
4. Arrêter l'entraînement de notre modèle lorsque l'accuracy n'augmente plus depuis un certain nombre d'époques pour éviter le gaspillage de ressources (mémoire, temps, GPU).

D'un côté plus positif, un point fort de notre approche était l'augmentation des données en appliquant de différentes transformations sur celles-ci. Ça nous a permis de balancer nos classes tout en obtenant des données supplémentaires fiables. De plus, le prétraitement que nous avons appliqué aux images nous semble efficace puisqu'il permet de normaliser et de standardiser les données ainsi que d'augmenter les contrastes des couleurs.

En conclusion, bien que le CNN semblait la meilleure option pour le traitement de nos images, c'était un modèle trop complexe pour la quantité de données que nous avions et un modèle moins complexe aurait été plus efficace pour la généralisation de nos données et moins propice au surapprentissage.

7. Références

Jalon 1 :

- L'implémentation de KNN et Parzen est tirée du laboratoire 2 de notre cours.
- L'implémentation de la régression logistique est tirée du laboratoire 4 de notre cours.

Jalon 2 :

- L'implémentation du modèle CNN et NN a été tirée du laboratoire 9 de notre cours.